Fundamentals of Algorithm Analysis and Design

Design and Analysis of Algorithms

Dhannya SM

Algorithm Design Technique

Design phase:

- Understand the problem
- Decide the computational means: Model the problem, Exact vs Approximate, Programming paradigm, if any
- Decide the data structure
- Design the algorithm
- Prove correctness

Algorithm Design Technique

Design phase:

- Understand the problem
- Decide the computational means: Model the problem, Exact vs Approximate, Programming paradigm, if any
- Decide the data structure
- Design the algorithm
- Prove correctness

Analysis phase:

- Analyze time complexity
- Analyze space complexity, if applicable
- If not practical, go back to design.

Algorithm Design Technique

Problem types:

- Sorting
- Searching
- String processing
- Graph problems
- Combinatorial problems
- Geometric problems

Analysis framework

- Defining input size
- Unit of running time, basic operations
- Count the number of times the algorithm's basic operation is executed on inputs of size n
- Ignores multiplicative constants and concentrates on the count's **order of growth**
- Order of growth means how the algorithm's time is going to vary based on change in size of the input.
- A difference in running times on really large inputs is what distinguishes efficient algorithms from inefficient ones

Analysis framework (cont.)

n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3.3	10^1	$3.3 \cdot 10^1$	10^2	10^3	10^3	$3.6 \cdot 10^6$
10^2	6.6	10^2	$6.6 \cdot 10^2$	10^4	10^6	$1.3 \cdot 10^{30}$	$9.3 \cdot 10^{157}$
10^3	10	10^3	$1.0 \cdot 10^4$	10^6	10^9		
10^4	13	10^4	$1.3 \cdot 10^5$	10^8	10^{12}		
10^5	17	10^5	$1.7 \cdot 10^6$	10^{10}	10^{15}		
10^6	20	10^6	$2.0 \cdot 10^7$	10^{12}	10^{18}		

Figure: How $f(n)$ varies with n

Analysis framework

- Worst-case, average-case, best-case
- Binary search vs Linear search

Asymptotic notations

To compare orders of growth, computer scientists use three notations.

- O **(big oh):** $O(g(n))$ is the set of all functions with a lower or same order of growth as $g(n)$
- Ω **(big omega):** $\Omega(g(n))$ is the set of all functions with a higher or same order of growth as $g(n)$
- Θ **(big theta):** $\Theta(g(n))$ is the set of all functions with the same order of growth as $g(n)$

Asymptotic notation: O (big Oh)[1]

A function $f(n)$ is said to be in $O(g(n))$, denoted $f(n) \in O(g(n))$, if $f(n)$ is bounded above by some constant multiple of $g(n)$ for all large n, i.e., if there exist some positive constant c and some non-negative integer n_0 such that

$$f(n) \leq cg(n), \ \forall \ n \geq n_0$$

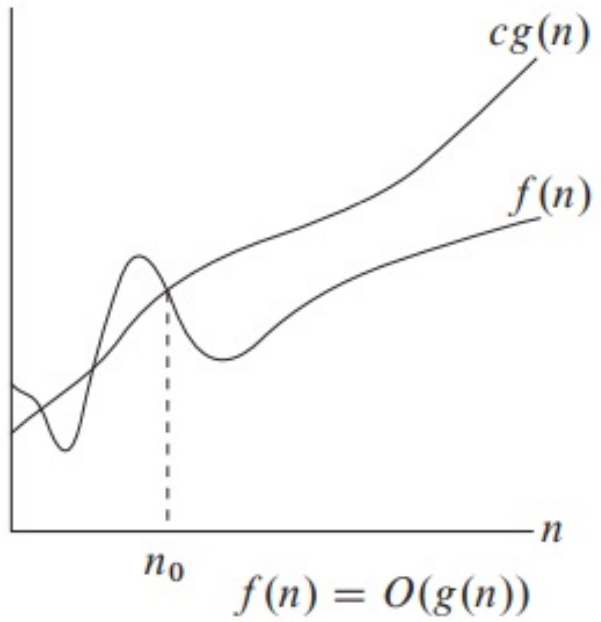


Figure: Big-Oh

Asymptotic notation: Ω (big Omega)[2]

A function $f(n)$ is said to be in $\Omega(g(n))$, denoted $f(n) \in \Omega(g(n))$, if $f(n)$ is bounded below by some constant multiple of $g(n)$ for all large n, i.e., if there exist some positive constant c and some non-negative integer n_0 such that

$$f(n) \geq cg(n), \ \forall \ n \geq n_0$$

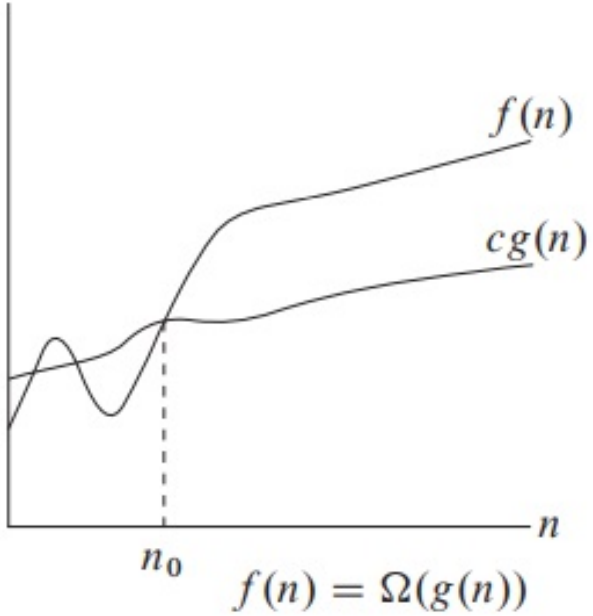


Figure: Big-Omega

Asymptotic notation: Θ (big Theta)[3]

A function $f(n)$ is said to be in $\Theta(g(n))$, denoted $f(n) \in \Theta(g(n))$, if $f(n)$ is bounded both above and below by some positive constant multiples of $g(n)$ for all large n, i.e., if there exist some positive constants c_1 and c_2 and some non-negative integer n_0 such that

$$c_1 g(n) \leq f(n) \leq c_2 g(n), \ \forall \ n \geq n_0$$

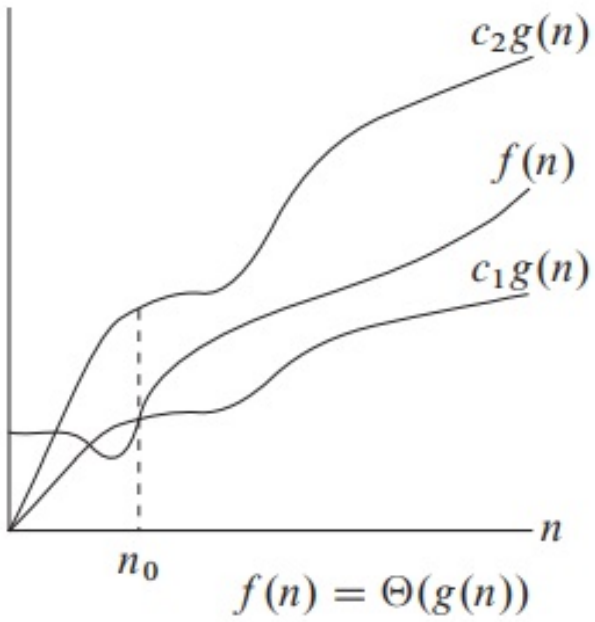
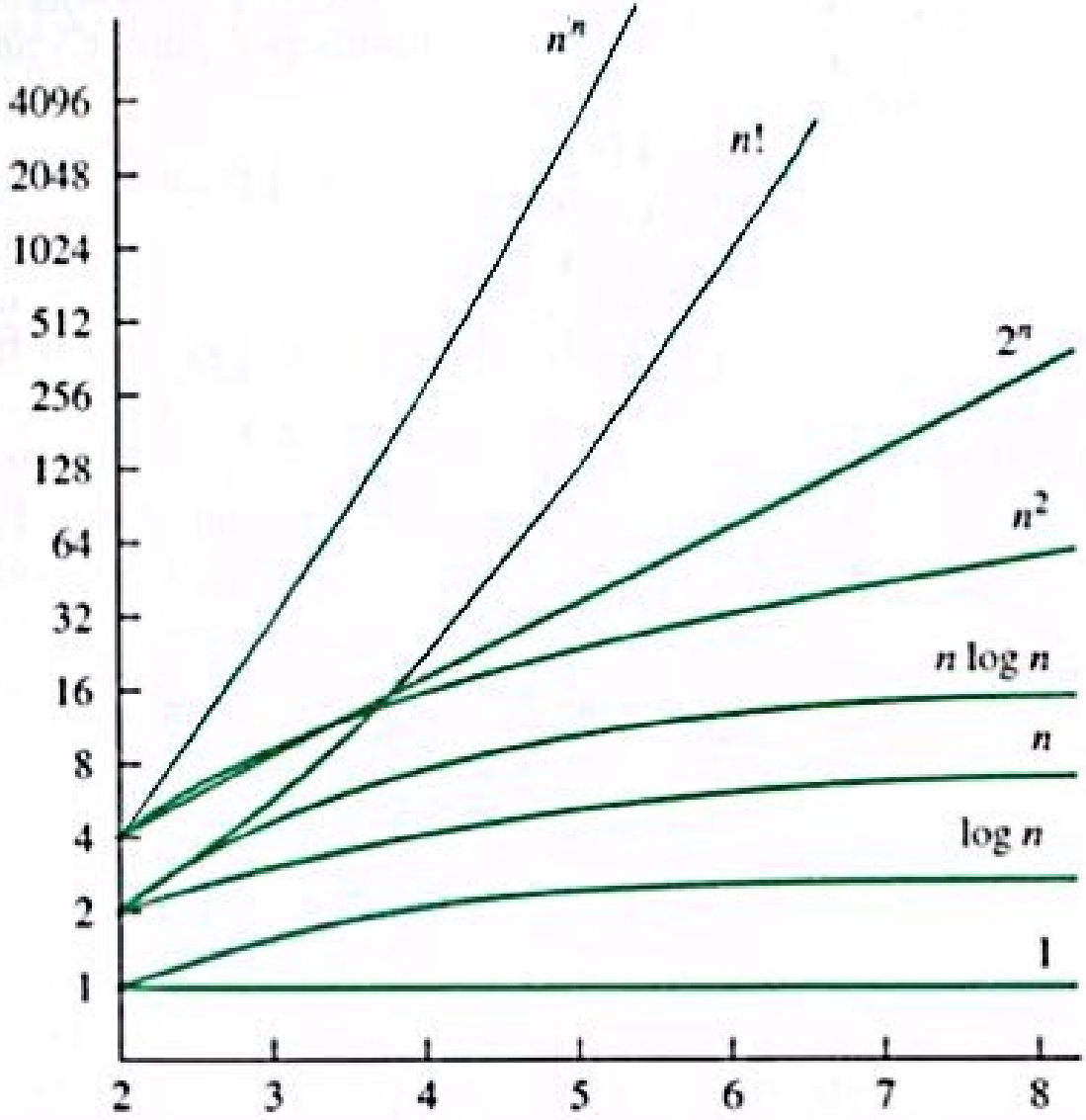


Figure: Big-Theta

Order of growth[4]



Using limits to compare orders of growth

$$\lim_{n\to\infty} \frac{t(n)}{g(n)} = \begin{cases} 0 & \text{implies that } t(n) \text{ has a smaller order of growth than } g(n), \\ c & \text{implies that } t(n) \text{ has the same order of growth as } g(n), \\ \infty & \text{implies that } t(n) \text{ has a larger order of growth than } g(n). \end{cases}$$

- When the limit is equal to 0 or c, then $t(n) \in O(g(n))$.
- When the limit is equal to c or ∞, $t(n) \in \Omega(g(n))$.
- When the limit is equal to c, then $t(n) \in \Theta(g(n))$.

Using limits to compare orders of growth

- To make the computation using limits easy, we use L'Hôspital's rule

$$\lim_{n \to \infty} \frac{t(n)}{g(n)} = \lim_{n \to \infty} \frac{t'(n)}{g'(n)}$$

Compare the orders of growth of $\frac{1}{2}n(n-1)$ and n^2.

Property of asymptotic notations

- If $t_1(n) \in O(g_1(n))$ and $t_2(n) \in O(g_2(n))$, then

$$t_1(n) + t_2(n) \in O(max(g_1(n), g_2(n)))$$

Summary

- Designing algorithm
  - Computational means
  - Data structure
  - Prove correctness
- Analysis of algorithm
  - Time and space complexity
  - Input size
  - Asymptotic notations